



1ST YEAR BASIC TRAINING IN CYBER SECURITY



INTRODUCTION TO OPERATING SYSTEMS 2 (SYST2)

Dr. Sassi BENTRAD

✉ : sassi.bentrad.enscs@gmail.com | | sassi.bentrad@enscs.edu.dz

LISCO Laboratory (Laboratoire d'Ingénierie des **S**ystèmes **C**omplexes)
University of Badji Mokhtar-Annaba (**UBMA**)

Basic Training in Cyber Security (1BT)
Formation de Base en Cyber-Sécurité (1FB)



CHAPTER



INTRODUCTION TO OPERATING SYSTEMS 2

GENERAL INTRODUCTION (5 %)



COURSE CONTENT

CHAPTER 1

- ❑ Introduction
- ❑ **Review** : Basic Computer Architecture
*Review **fundamental concepts and structures of computer systems.***
such as :
 - ✓ **CPU** – Central Processing Unit
 - ✓ **Memory Hierarchy** – Registers, Cache memory, Main memory (RAM), Secondary storage
 - ✓ **Buses** – Communication between components
 - ✓ **I/O Systems** – Input devices, Output devices, Network interfaces, Interrupts and I/O management
 - ✓ **Storage Systems** – Hard Disk Drives (HDDs), Solid-State Drives (SSDs)
 - ✓ **Current Trends** – Multicore and Multiprocessing systems, Parallel computing, Specialized architectures (GPU, FPGA, AI accelerators), ...
- ❑ Summary & Key Takeaways

❑ Introduction & Objectives

Objectives :

By the end of this chapter, students will be able to:

- **Understand** the fundamental principles of computer architecture.
- **Identify and describe** the key components of a computer system (CPU, memory, buses, I/O, and storage).
- **Explain** how **hardware architecture** influences **operating system** design and functionality.
- **Analyze** the **interaction** between **hardware components** and **system software**.

❑ Introduction & Objectives

Why does it matter? *Why study Computer Architecture?*

- **Performance** – Affects execution speed and efficiency.
- **Efficiency** – Optimizes hardware and energy use.
- **System Design** – Forms the foundation of embedded systems, supercomputers, and modern architectures.

Key Topics Covered :

- ✓ **CPU** – Central Processing Unit
- ✓ **Memory Hierarchy** – Registers, Cache memory, Main memory (RAM), Secondary storage
- ✓ **Buses** – Communication between components
- ✓ **I/O Systems** – Input devices, Output devices, Network interfaces, Interrupts and I/O management
- ✓ **Storage Systems** – Hard Disk Drives (HDDs), Solid-State Drives (SSDs)
- ✓ **Current Trends** – Multicore and Multiprocessing systems, Parallel computing, Specialized architectures (GPU, FPGA, AI accelerators), ...

❑ What is a Computer ?

- A **computer** is an electronic device that can process and store information.
- It can perform calculations, manipulate data, and execute a set of instructions to accomplish specific tasks.
- It consists of **hardware** (*physical components*) and **software** (*programs and operating systems*).
- Computers follow the **Von Neumann architecture**, processing instructions sequentially, and they come in various forms like *Desktops*, *Laptops*, and *Embedded Systems*.
- ...

❑ What is a Computer ?

• Core Components of a Computer

1. Hardware: The physical parts ...

- **Central Processing Unit (CPU):** Control Unit (CU) and Arithmetic Logic Unit (ALU) (*as in Von Neumann architecture*).
- **Memory:** RAM (Random Access Memory); ROM (Read-Only Memory); Cache.
- **Storage Devices:** Long-term data storage (e.g., HDD, SSD, **Cloud**).
- **Input/Output (I/O) Devices:** Allow interaction (e.g., Keyboard, Mouse, Monitor).
- **Buses:** Communication pathways (**Address, Data, Control**) connecting components.

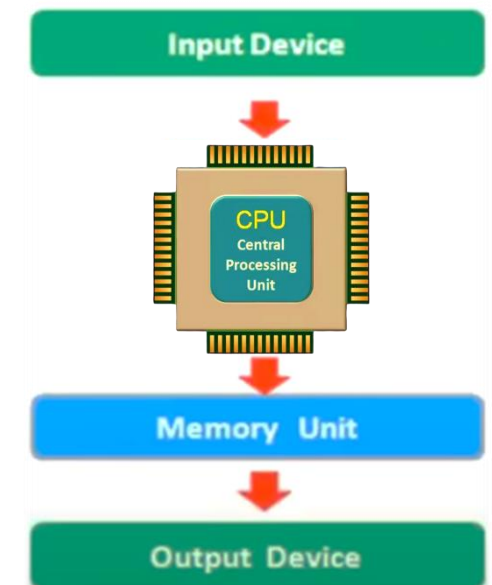
2. Software: Instructions ...

- **Operating System (OS):** Manages hardware and software resources ...
- **Applications:** Programs for specific tasks ...

❑ What is a Computer ?

• How a Computer Works ?

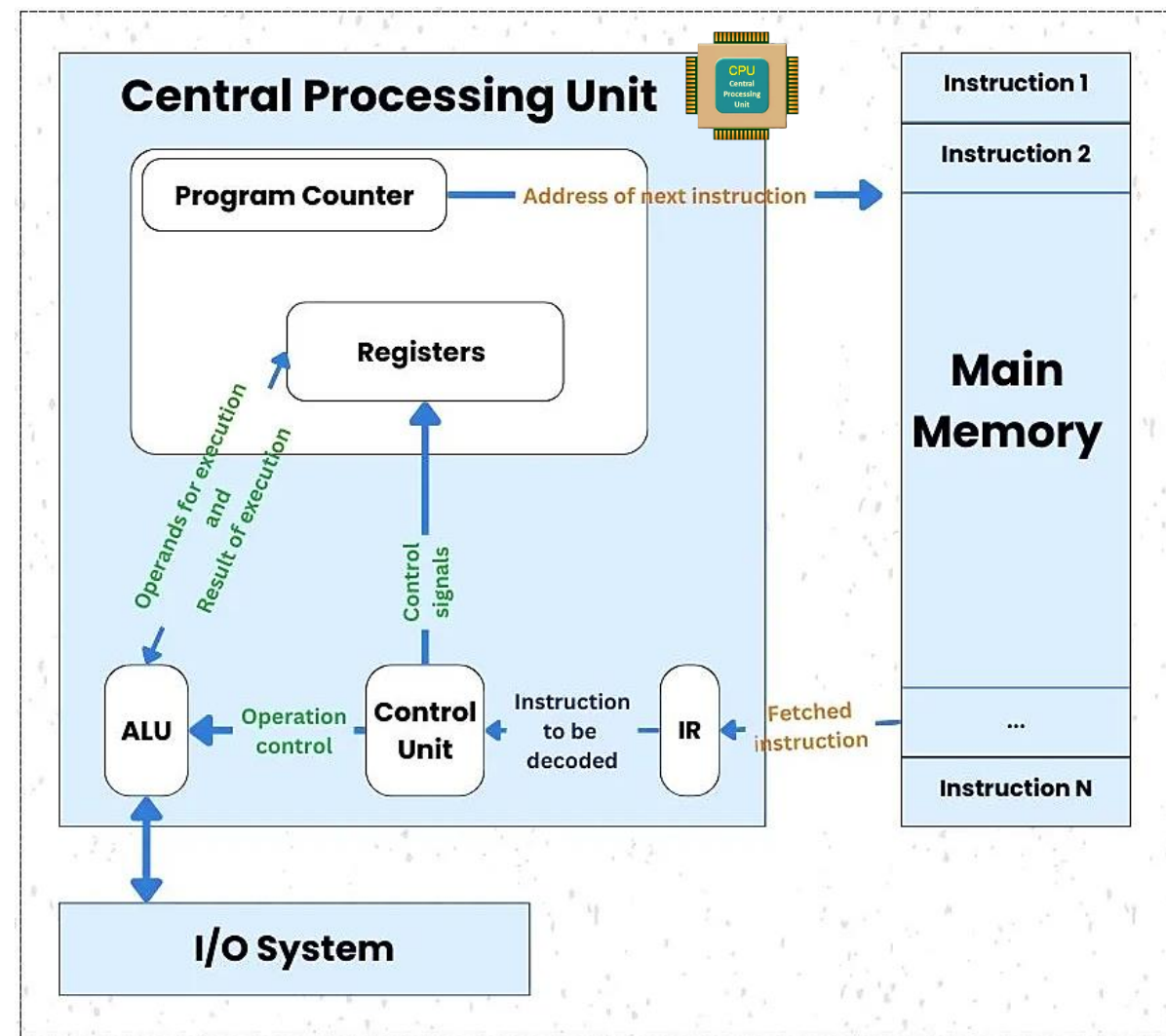
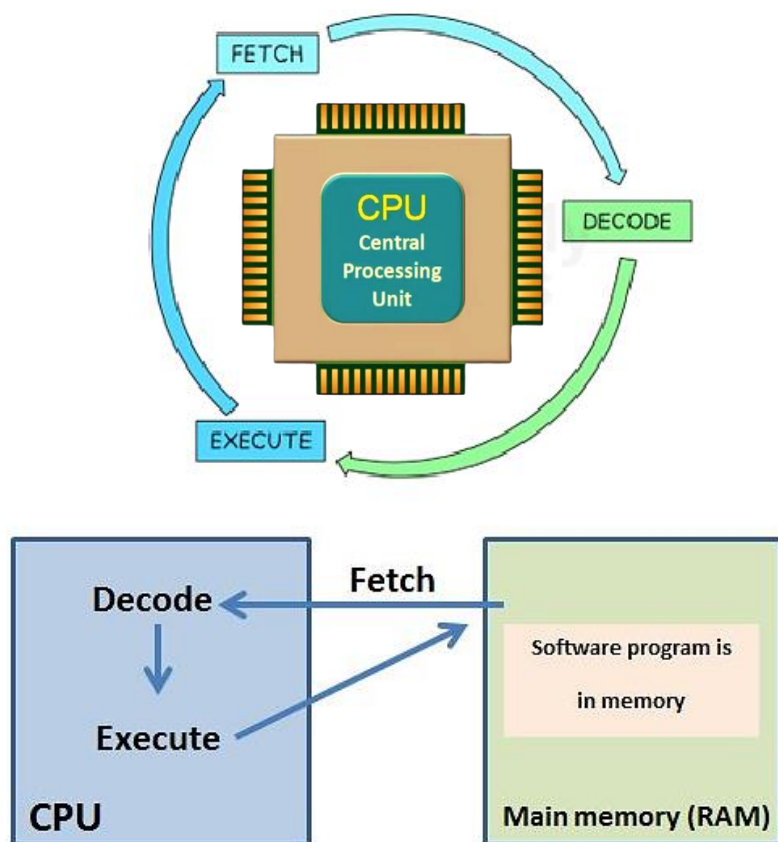
1. **Input:** Data / Instructions are entered via I/O devices (keyboard, mouse, sensors).
2. **Processing:**
 - The CPU **fetches** instructions from Memory.
 - It **decodes** them and **executes** operations (*calculations, data movement*).
 - Follows the **Fetch-Decode-Execute cycle** (*Von Neumann model*).
3. **Storage:** Data is saved temporarily (RAM) or permanently (Hard drive).
4. **Output:** Results are displayed (e.g., on a monitor, printer, speakers) or saved for later use.



❖ REVIEW : BASIC COMPUTER ARCHITECTURE

❑ What is a Computer ?

• How a Computer Works ?



Fetch, Decode and Execute Cycle ♻️

❑ What is a Computer ?

• Key Characteristics

- ✓ **Programmability:** Can perform different tasks by changing software.
- ✓ **Automation:** Executes tasks without constant human intervention.
- ✓ **Speed:** Processes billions of operations per second (e.g., modern CPUs).
- ✓ **Accuracy:** Reliable if hardware/software is functioning correctly.
- ✓ **Versatility:** Used for calculations, communication, entertainment, AI, ...

• Types of Computers

1. General-Purpose:

- ✓ **Personal Computers** (PCs): Desktops, laptops.
- ✓ **Servers:** Manage networks and data for multiple users.

2. Specialized:

- ✓ **Supercomputers:** Solve complex scientific problems (e.g., weather modeling, ...).
- ✓ **Embedded Systems:** Built into devices (e.g., smartphones, cars, ...).

3. Analog vs. Digital:

Most modern computers are **digital** (process discrete values : **0s** and **1s**).

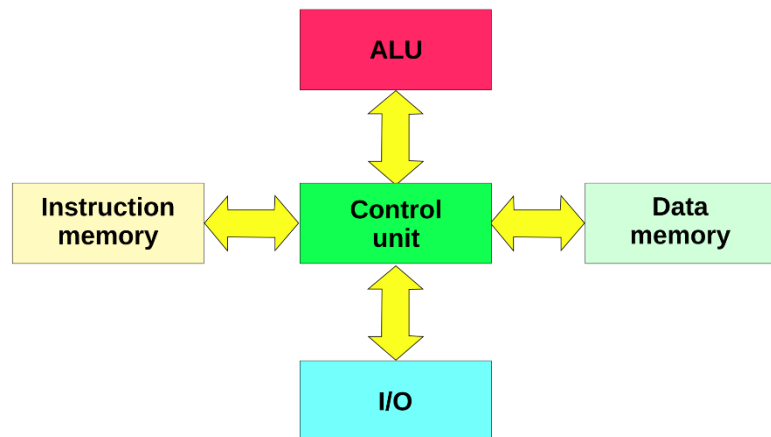
❑ Comprehensive Overview of Computer Organization and Architecture

- **Computer Architecture** : The conceptual design and structure of a computing system.

- **Main Types:**

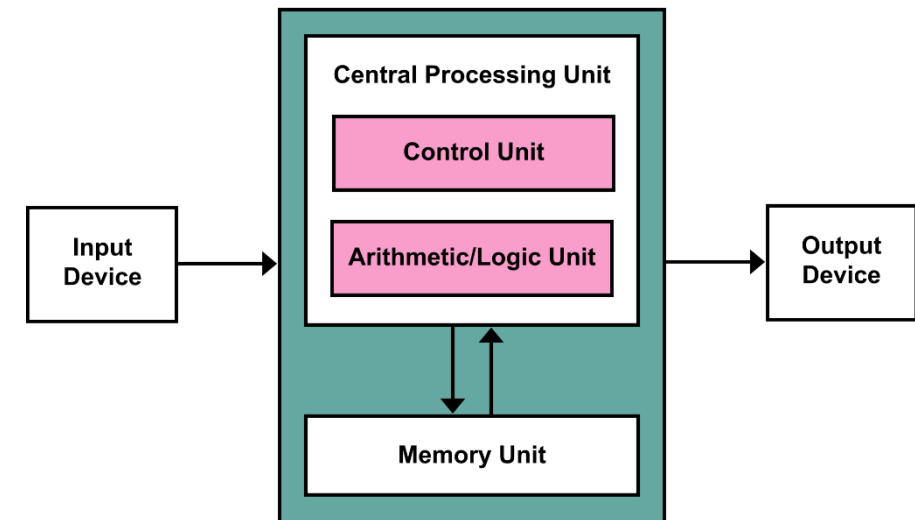
- ✓ **Harvard Architecture:**

- ✓ *Separate memory for instructions & data.*



- ✓ **Von Neumann Architecture:**

- Unified memory for instructions & data.*



❑ Comprehensive Overview of Computer Organization and Architecture

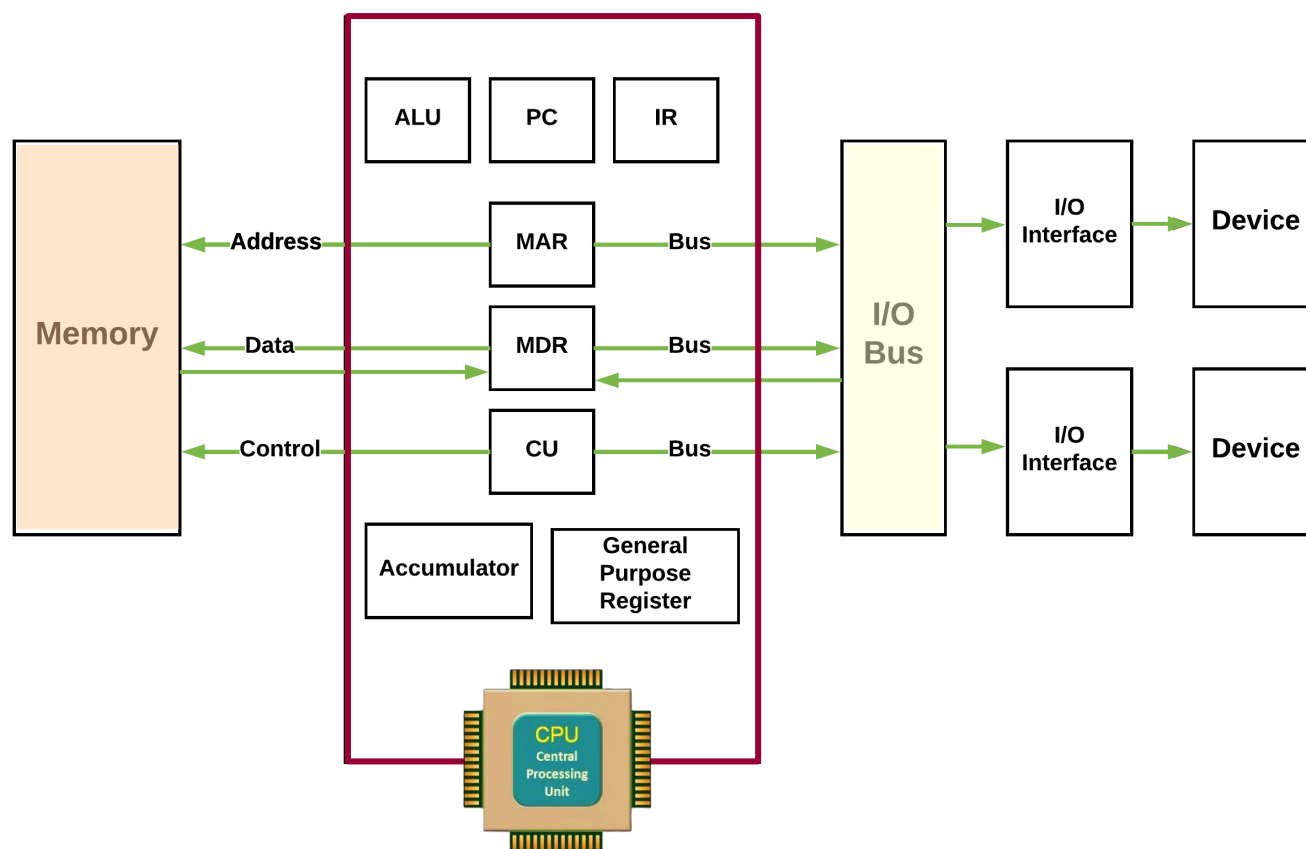
- **Computer Architecture** : The conceptual design and structure of a computing system.
- **Main Types:**
 - ✓ **Harvard Architecture:**
✓ *Separate memory for instructions & data.*
 - ✓ **Von Neumann Architecture:**
Unified memory for instructions & data.
- **Key performance factors :**
 1. **Clock Speed** ⌚ – Determines how fast a CPU executes instructions. Measured in GHz.
 2. **Parallelism** 🔄 – Involves multiple processors or cores working simultaneously.
 3. **Memory Hierarchy** 🧠 – Optimizes data access speed through a structured hierarchy, including **registers**, **caches**, **RAM**, and **storage devices** for efficient performance.

❑ Comprehensive Overview of Computer Organization and Architecture

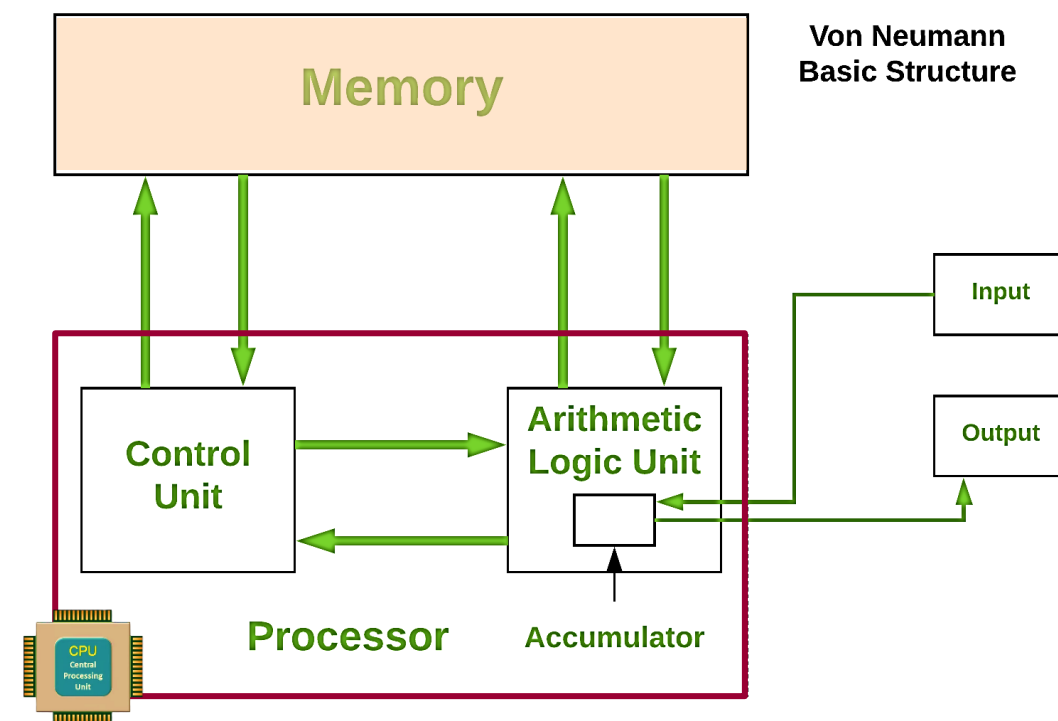
Harvard Architecture vs. Von Neumann Architecture

Aspect	Von Neumann Architecture	Harvard Architecture
Memory	Single memory for data and instructions	Separate memory for data and instructions
Bus	Single bus for data and instructions	Separate buses for data and instructions
Performance	Slower due to potential bottlenecks	Faster due to simultaneous access
Complexity	Simpler and cheaper to implement	More complex and costly
Flexibility	Highly flexible for general-purpose computing	Less flexible, specialized for specific tasks
Use Cases	General-purpose computers, laptops, desktops	Embedded Systems, Digital Signal Processors (DSPs), Microcontrollers (MCUs)

❑ Comprehensive Overview of Computer Organization and Architecture



Basic CPU structure, illustrating ALU



Von Neumann architecture

□ Von Neumann Architecture

- Currently, the **architecture of most computers**.
 - It derives its name from the eminent Hungarian-American mathematician **John von Neumann (1903-1957)**;
 - The program instructions, its data structures, and peripheral components **share the same memory area**.
- 
- **The key components :**
 1. **Processing Unit (CPU):** Includes the Control Unit (CU) and the Arithmetic Logic Unit (ALU).
 2. **Memory:** Stores both data and instructions (a key characteristic of this model).
 3. **Bus System:** Facilitates communication between components (data bus, address bus, and control bus).
 4. **Input/Output (I/O) Devices:** Allow interaction with external systems.

□ Von Neumann Architecture

- **Key Characteristics :**

1. **Single Memory for Data and Instructions:** Both data and program instructions are stored in the same memory.
2. **Shared Bus:** A single bus is used for transferring data, addresses, and control signals, which can limit performance.
3. **Sequential Execution:** Instructions are executed one at a time in a sequential manner.

□ Von Neumann Architecture

Advantages :

- **Simplified Design:** Uses a single memory for data and instructions, reducing hardware complexity.
- **Cost-Effective:** Lower production costs due to fewer components.
- **Flexibility:** Can run various programs and makes it suitable for general-purpose computing.
- **Ease of Programming:** Unified memory structure simplifies software development.
- **Widely Adopted:** Forms the foundation of most modern computers hence, ensures widespread compatibility.

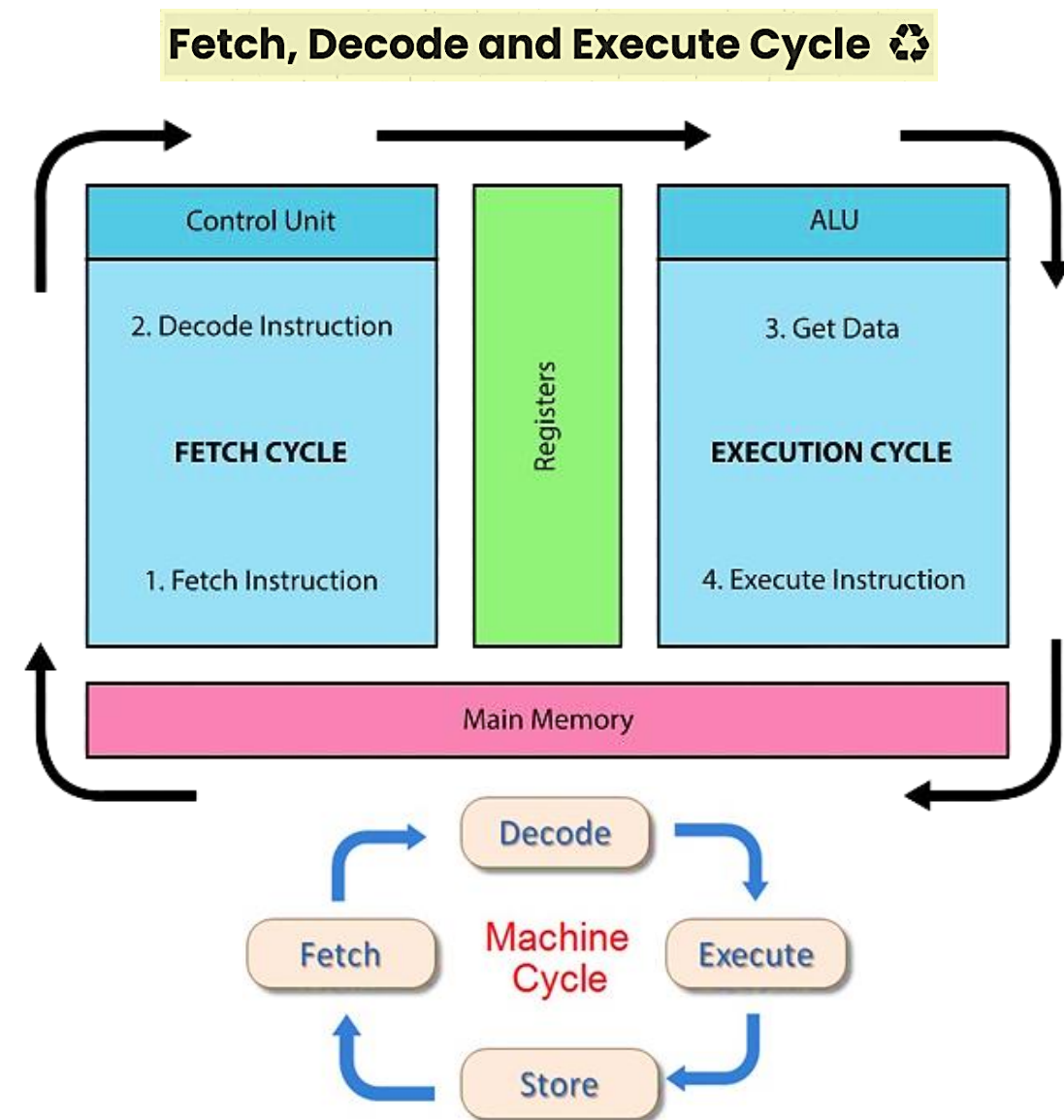
Limitations :

- **Memory Bottleneck:** Shared memory slows down data and instruction transfer.
- **Sequential Processing:** Cannot process data and instructions simultaneously.
- **Scalability Issues:** Struggles with high-performance tasks requiring rapid memory access.
- **Energy Inefficiency:** Frequent memory access increases power consumption.
- **Latency:** Data and instruction fetch delays reduce overall system efficiency.

❑ Von Neumann Architecture

The 5 stages of the Von Neumann architecture :

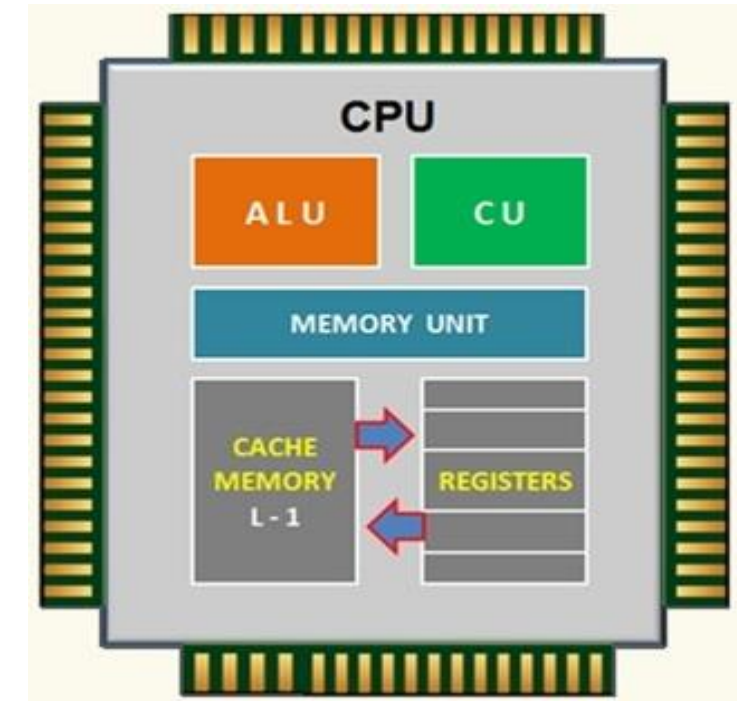
1. **Fetch**: The CPU retrieves an instruction from memory.
2. **Decode**: The instruction is translated into signals the computer understands.
3. **Execute**: The CPU performs the instruction (e.g., calculations or data movement).
4. **Memory Access**: Data is read from or written to memory if needed.
5. **Write Back**: Results are saved back to memory or registers.



❑ Central Processing Unit (CPU) & Its Components

The **CPU** is the **brain** of the computer system that provides **Computational** and **Decision** making power.

- **Control Unit (CU)**: Directs operation execution.
- **Arithmetic Logic Unit (ALU)**: Performs calculations and logical operations.
- **Registers**: Small, high-speed storage inside the CPU.
- **Execution Cycle**: **Fetch** ⇒ **Decode** ⇒ **Execute** ⇒ **Store**.



❑ Registers in a Computer

Registers are very fast internal memory units located within the processor (CPU). They play a crucial role in processing instructions and data by acting as temporary storage for information needed to execute operations.

1. Characteristics of Registers

- ✓ **Very fast:** Faster than RAM and even cache memory.
- ✓ **Small capacity:** Usually a few bytes to a few dozen bytes **(8, 16, 32, 64 bits)**.
- ✓ **Directly accessible by the processor:** They minimize data access time.
- ✓ **Temporary storage:** Holds data only during the execution of an instruction.

The number of registers in a **CPU** depends on its **architecture** (**x86**, **x86-64**, **ARM**, **RISC-V**, etc.) and the type of registers (**general-purpose** vs. **specialized**). Here's an overview:

Architecture	General-Purpose Registers	Vector Registers	Other Registers
Intel x86-64	16 (RAX, RBX, ...)	16 XMM / YMM / ZMM	6 segment registers, PC, SP, Flags
ARMv8-A	31 (X0–X30)	32 NEON (128-bit)	PC, SP, Flags
RISC-V RV64GC	32 (x0–x31)	32 vector registers (optional)	PC, SP, Flags

❑ Registers in a Computer

2. Types of Registers

A. General-Purpose Registers (GPRs)

- 📌 Used to store temporary data during instruction execution.
- 📌 Examples: **AX**, **BX**, **CX**, **DX** (in **x86** architectures).

B. Specific Registers

- ◆ **Instruction Register (IR)** – *Holds the current instruction being executed.*
- ◆ **Program Counter (PC)** – *Stores the address of the next instruction to be executed.*
- ◆ **Stack Pointer (SP)** – *Points to the top of the stack in memory.*
- ◆ **Memory Address Register (MAR)** – *Holds memory addresses for fetching instructions or data.*
- ◆ **Memory Data Register (MDR)** – *Temporarily stores data being transferred to/from memory.*
- ◆ **Status Register / Flags Register (FLAGS, EFLAGS, RFLAGS)** – *Indicates the processor's status (e.g., overflow, zero, negative flags).*

❑ Registers in a Computer

3. Importance of Registers in Program Execution

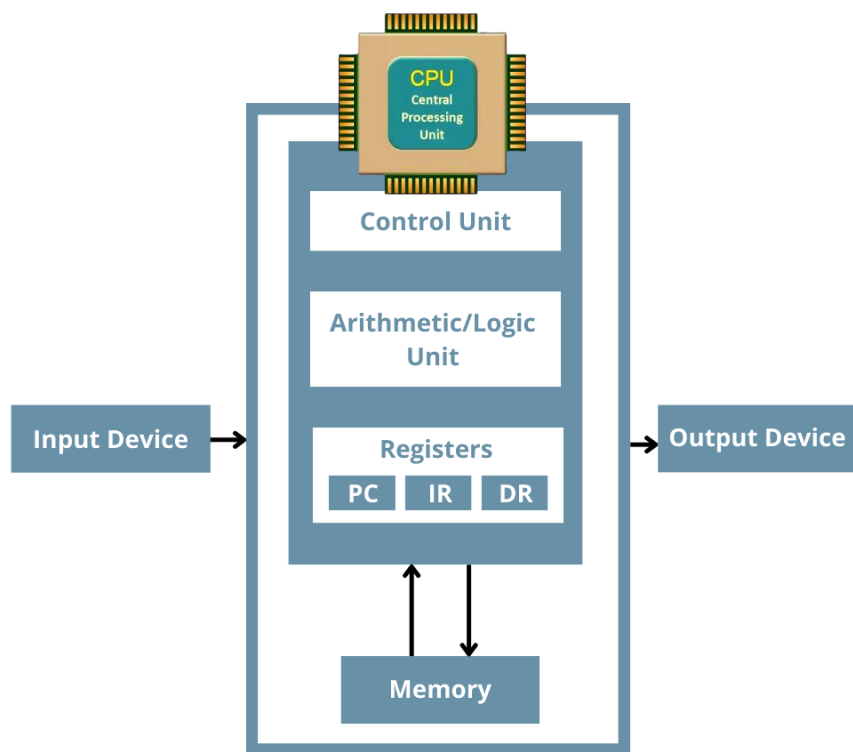
Registers play a crucial role in program execution by providing **high-speed storage** for **immediate data processing**.

Their importance includes:

- **Fast Data Access** – are the fastest memory in the CPU, reducing latency in instruction execution.
- **Efficient Instruction Execution** – store operands, addresses, and intermediate results, speeding up arithmetic and logic operations.
- **Control Flow Management** – help manage execution flow and function calls (like **PC** and **SP**)
- **Reduced Memory Access Bottleneck** – by minimizing frequent access to slower RAM, registers improve overall system performance.
- **Optimized CPU Operations** – enhance multitasking and complex computations (e.g., **Specialized registers**).

❑ Registers in a Computer

- **Registers** are used to store **addresses** or **intermediate calculation results**.
- Most assembler commands use registers $\Rightarrow$ it's important to know their functions.



A graphical illustration of registers in a CPU

Register Type	Function
General-Purpose	Flexible storage for operations and data management.
Accumulator	Stores results of arithmetic and logical operations.
Instruction	Keeps track of the current instruction being executed.
Program Counter	Maintains the address of the next instruction to execute.
Status	Flags to indicate operational states (e.g., zero, overflow).

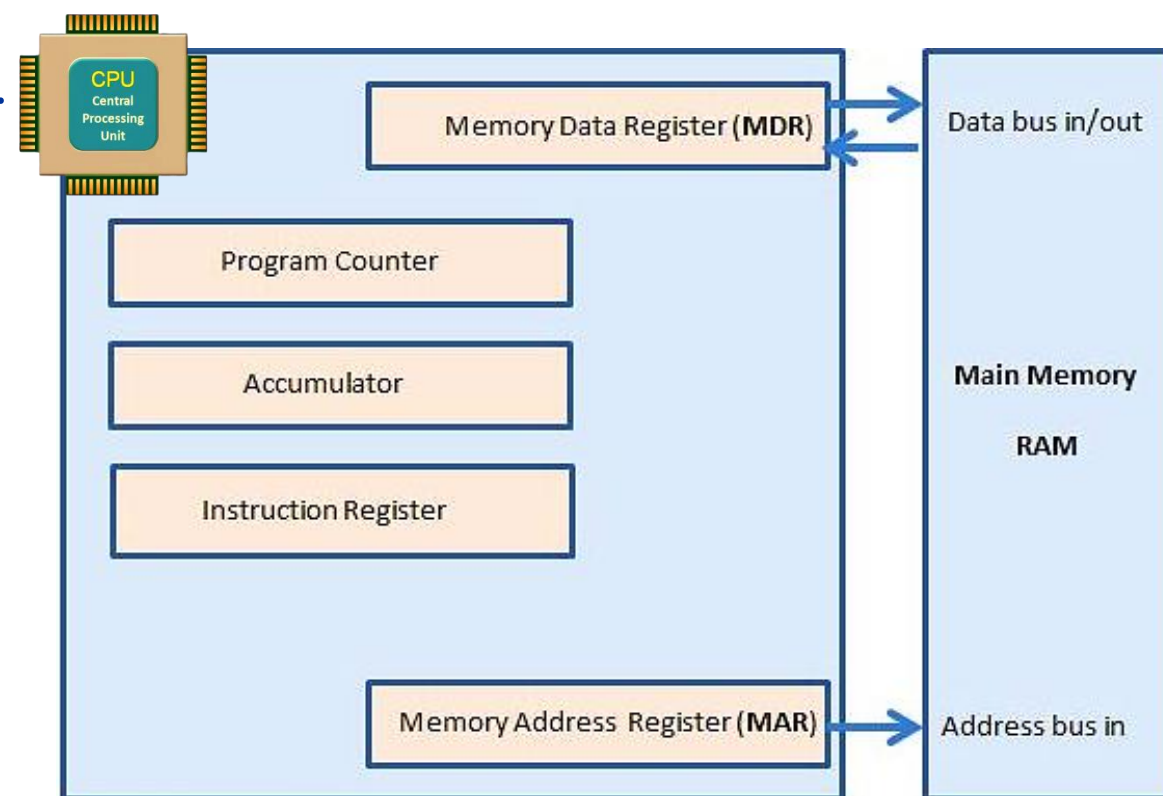
❑ Registers in a Computer / CPU Machine Cycle

A number of registers inside the **CPU** are involved with the cycle **Fetch Decode Execute**.

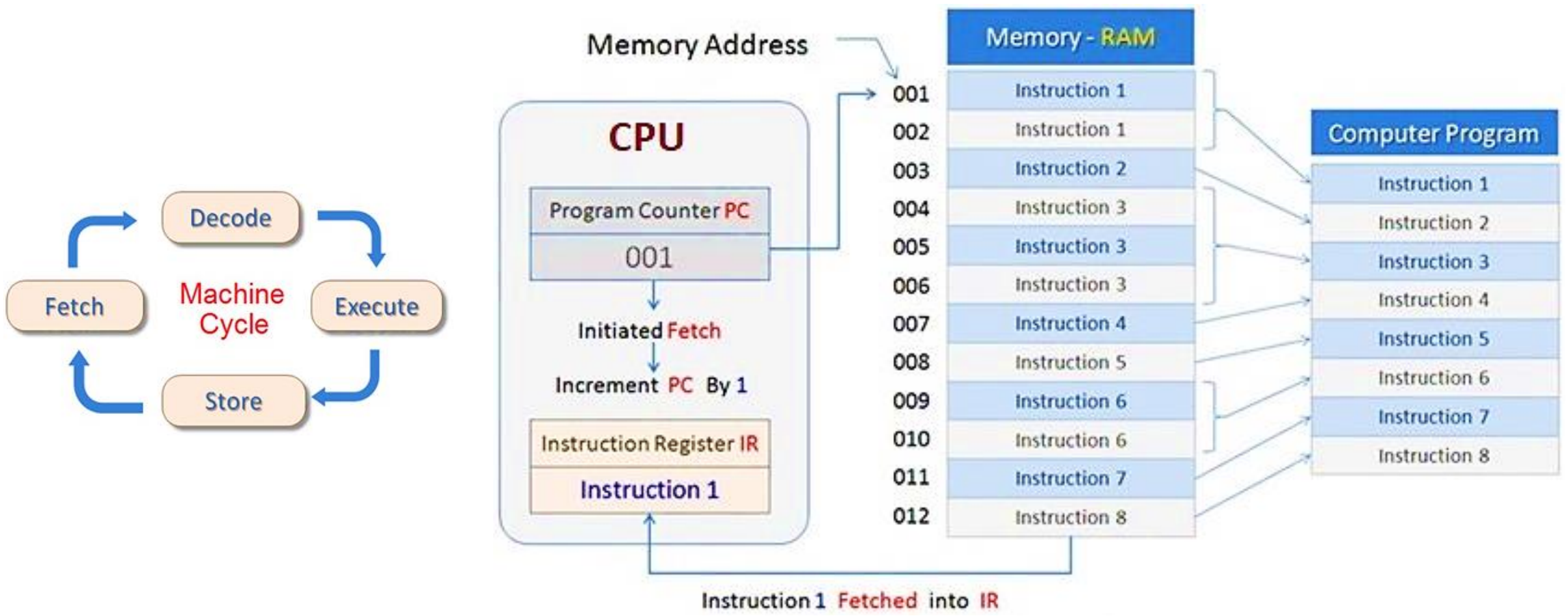
The registers are :

- **MDR** (Memory Data Register)
- **MAR** (Memory Address Register)
- **PC** (Program Counter)
- **Accumulator**
- **IR** (Instruction Register)

We will next explain how these registers are used in each stage of the **CPU Machine Cycle**.



❑ CPU Machine Cycle



Program-Counter Register and Instruction-Register

❑ 8086 vs. x86 vs. x86-64 (x64)

8086 Architecture (16-bit)

- **Introduced:** 1978 by Intel.
- **Processor type:** 16-bit.
- **Memory addressing:** Can address **1 MB** of memory.
- **Registers:** Uses 16-bit registers like AX, BX, CX, DX, SP, BP, SI, DI.
- **Mode:** Real Mode (no memory protection or virtual memory).
- **Instructions:** Basic instruction set for simple operations.
- **Compatibility:** Not compatible with newer 32-bit or 64-bit software.
- **Common use:** Used in the early IBM PCs.

8086 Assembly (16-bit)

- Uses **16-bit registers** (AX, BX, CX, DX, SP, BP, SI, DI).
- Memory segmentation with segment registers (**CS, DS, SS, ES**).
- Limited instruction set, supports only **real mode** execution.
- **Example:** Simple addition program in 8086 assembly using **MASM** syntax:

```
MOV AX, 2    ; Load 2 into AX
ADD AX, 3    ; Add 3 to AX
MOV BX, AX   ; Store result in BX
```

❑ 8086 vs. x86 vs. x86-64 (x64)

x86 Architecture (32-bit)

- **Introduced:** 1985 with the Intel 80386.
- **Processor type:** 32-bit.
- **Memory addressing:** Can address **4 GB** of memory (using 32-bit addresses).
- **Registers:** Uses 32-bit registers like EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI.
- **Mode:** Supports Real Mode (for backward compatibility) and Protected Mode (for more advanced features like memory protection and multitasking).
- **Instructions:** Extended set, including **SSE** (Streaming SIMD Extensions) and **MMX** (MultiMedia Extensions) for more efficient computing.
- **Common use:** Dominated desktop computers for decades (Pentium, Core 2 Duo, etc.), compatible with most software from the 1990s and 2000s.

x86 Assembly (IA-32, 32-bit)

- Uses **32-bit registers** (EAX, EBX, ECX, EDX, ESP, EBP, ESI, EDI).
- Supports **protected mode**, virtual memory, and privilege levels.
- Introduces **SSE/MMX** instructions for SIMD operations.
- **Example:** Addition program in x86 assembly (32-bit **NASM**):

```
section .text
    global _start

_start:
    mov eax, 2    ; Load 2 into EAX
    add eax, 3    ; Add 3 to EAX
    mov ebx, eax  ; Store result in EBX
    mov eax, 1    ; syscall: exit
    int 0x80      ; call kernel
```

❑ 8086 vs. x86 vs. x86-64 (x64)

x86-64 (x64) Architecture (64-bit)

- **Introduced:** 2003 by AMD (AMD64), adopted by Intel 64.
- **Processor type:** 64-bit.
- **Memory addressing:** Can theoretically address **16 exabytes** of memory, though operating systems and hardware currently limit it to a few terabytes.
- **Registers:** Uses 64-bit registers like RAX, RBX, RCX, RDX, RSP, RBP, RSI, RDI, and additional registers like R8–R15.
- **Mode:** Includes a **long mode** (64-bit mode) for full 64-bit operations. Compatible with 32-bit and 16-bit code (in compatibility mode).
- **Instructions:** More powerful and efficient with 64-bit wide data paths and new instructions (e.g., for better performance in mathematical and multimedia applications).
- **Common use:** Modern desktop and server systems; most contemporary software is written for 64-bit (Windows, Linux, macOS).

x86-64 Assembly (64-bit)

- Uses **64-bit registers** (RAX, RBX, RCX, RDX, RSP, RBP, RSI, RDI).
- Supports **long mode** and a larger instruction set.
- More general-purpose registers (**R8–R15** added).
- Improved calling conventions (arguments passed via registers instead of stack).
- **Example:** Addition program in x86-64 assembly (**NASM**):

```
section .text
global _start

_start:
    mov rax, 2    ; Load 2 into RAX
    add rax, 3    ; Add 3 to RAX
    mov rbx, rax  ; Store result in RBX
    mov rax, 60   ; syscall: exit
    xor rdi, rdi  ; return 0
    syscall      ; call kernel
```

❑ 8086 vs. x86 vs. x86-64 (x64) / Key Differences

Feature	8086 (16-bit)	x86 (32-bit)	x86-64 (64-bit)
Processor Type	16-bit	32-bit	64-bit
Registers	AX, BX, CX, DX (16-bit)	EAX, EBX, ECX, EDX (32-bit)	RAX, RBX, RCX, RDX, etc. (64-bit)
Memory Addressing	1 MB	4 GB	16 exabytes (theoretical)
Compatibility	Runs 16-bit code only	Backward compatible with 16-bit code	Backward compatible with 32-bit and 16-bit
Common Usage	Early PCs (IBM PC XT)	Widely used from the 1990s to 2010s	Modern systems (PCs, servers)
Performance	Limited by 16-bit registers	Improved with 32-bit operations	Enhanced performance, supports 64-bit data paths

❑ 8086 vs. x86 vs. x86-64 (x64) / Summary

- These terms refer to **different CPU architectures**, and while they share similarities, each represents a significant evolution in computing hardware.
- **8086**: A legacy 16-bit processor, foundational but limited by modern standards.
- **x86**: A 32-bit processor with a much larger memory space, supporting modern OS and applications for many years.
- **x86-64 (x64)**: The modern 64-bit architecture, offering vastly improved memory addressing, performance, and compatibility with both older and newer software.

❑ Memory (*Memory Hierarchy*)

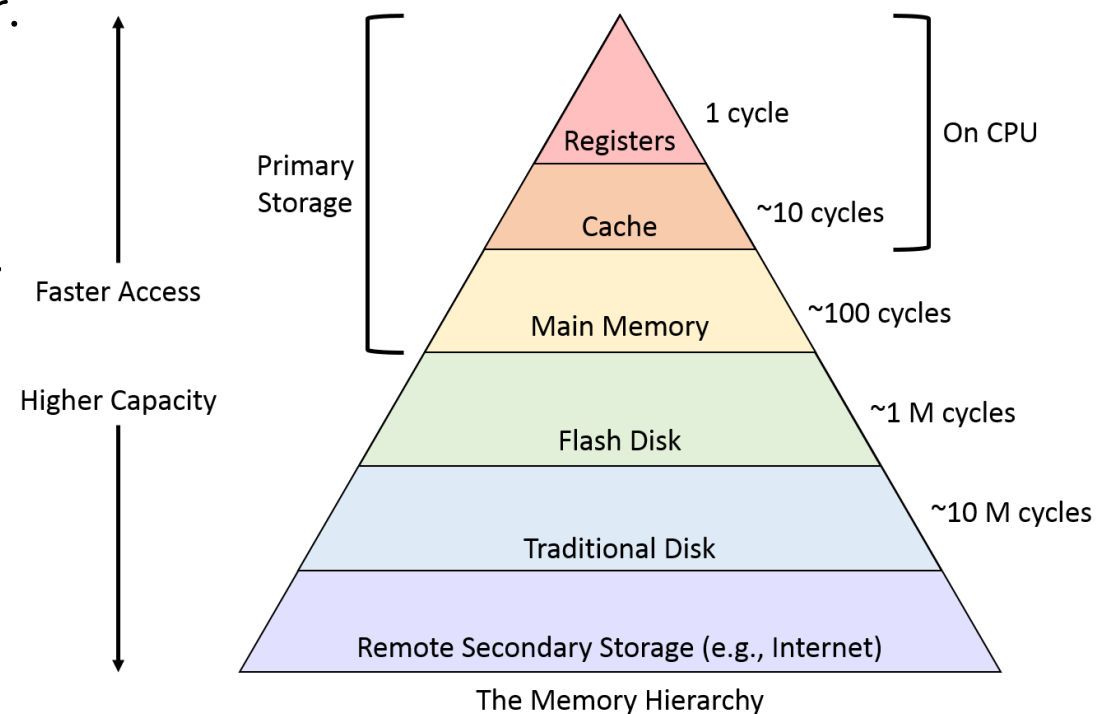
- **Registers**: Small, high-speed storage inside the CPU.
- **Cache Memory**: Faster than RAM, closer to CPU.
- **Main Memory (RAM)**: Volatile, fast access.
- **Secondary Storage (HDD/SSD)**: Non-volatile, slower.
- **Tertiary Storage** (Optical Discs, Magnetic tapes) : Used for backup and archival purposes.
- **Cloud Storage & Virtual Memory**: Extended storage.

Storage Systems :

- **Primary** Storage: RAM, Cache (Volatile, fast).
- **Secondary** Storage: HDD, SSD (Persistent, slower).
- **Tertiary** Storage: Magnetic tapes, archives.
- **Cloud** Storage: Remote access and backup.

SOURCE :

https://www.cs.swarthmore.edu/~kwebb/cs31/f18/memhierarchy/mem_hierarchy.html



❑ Memory (*Memory Word*)

- Only the **CPU** can modify the state of the memory. Information is retained until the power is cut off.
- **Possible operations:**
 - ✓ **Write:** The processor provides a value and an address, and the value is stored at the specified location.
 - ✓ **Read:** The processor requests the value stored at the location indicated by its address.
- Read and write operations involve multiple bytes in memory (*Memory Word*).
- The **size of a memory word** depends on the **processor architecture** :
 - ✓ **8-bit processors** → 8-bit memory word (Intel 8051)
 - ✓ **16-bit processors** → 16-bit memory word (Intel 80286, Texas Instruments MSP430)
 - ✓ **32-bit processors** → 32-bit memory word (Intel 80386, Intel 80486, Intel Pentium, ARM Cortex-A7)
 - ✓ **64-bit processors** → 64-bit memory word (Intel Core i7, AMD Ryzen, Apple M1,)
- The **memory word size** affects how much data can be processed in a single operation and the maximum addressable memory.
- Each generation improves **processing power**, **memory access**, and **overall system performance**.

❑ Buses System (*Data Transfer Mechanism*)

- **Buses** are communication pathways used to transfer data between different components of a computer system (**CPU, Memory, and I/O devices**).
- **Types of Buses :**
 - **Address Bus:** Carries memory addresses from the CPU to other components.
 - **Data Bus:** Transfers actual data between components.
 - **Control Bus:** Coordinates operations (sends control signals to manage operations. e.g., read/write signals).
- **Buses are critical for communication in a computer system.**
 - **Bus Width** affects data transfer speed.
 - **Efficient data flow** ensures optimal system operation and performance.

❑ Input/Output (I/O) System

- The **I/O system** enables communication between the CPU and external devices.
- It manages data exchange between the computer and peripherals.
- It is crucial for system functionality, impacting performance and user experience.
 - ✓ **I/O Devices**: Keyboards, mouse, displays, scanner, printers, speakers, etc.
 - ✓ **I/O Controllers**: Manage communication with CPU.
 - ✓ **Interrupts & Polling**: techniques used by CPUs to handle I/O requests from devices.
 - ✓ **Direct Memory Access (DMA)**: High-speed data transfer without CPU intervention for efficiency.

❖ SUMMARY & KEY TAKEAWAYS

□ ...

Any questions or clarifications ? *Let's discuss !*

- Q 1 : ...

- ...

THANK YOU for your attention!



Questions ?



For more information about my research works, **Contact Information:**

Dr. Sassi BENTRAD

LISCO Laboratory : <http://lisco.univ-annaba.dz/>

☎ : +213 ...

✉ : sassi.bentrad.enscs@gmail.com || sassi.bentrad@enscs.edu.dz

in : www.linkedin.com/in/sassi-bentrad/

🏠 : <http://www.bentrad-sassi.sitew.com/>

ORCID
Connecting Research
and Researchers

iD : orcid.org/0000-0002-7458-8121

RESEARCHERID : A-9442-2013

SCOPUS Author ID : 44461052600

